µInference

A verifiable minimum inference engine for LLMs

692 lines of trusted code.

Same output on seven different systems, bit for bit.

Runs on seL4, a formally verified microkernel.

SL5 Task Force · July 2026

What it is

A small LLM inference engine built to be checked.

Small

692 lines of code do the inference.

No libc. No libm. No heap.

No threads.

Reproducible

Same weights and prompt give the same

logits on every system

tested.

Isolated

Runs as one seL4 protection domain.

No device access. No Linux.

Bounded

Memory use is fixed at build time.

4.1 MB. No input can

change it.

What it is for

To check that some other system ran a model correctly, you have to run it again yourself.

That means you need an engine you trust more than the one you are checking.

Existing engines are too large to trust and do not give the same answer twice.

Why other engines are not reproducible

The cause is batching, not floating-point error.

vLLM, SGLang

They batch requests together.

Batch size changes how sums are split up.

Batch size depends on how busy the server is.

1000 runs of one prompt gave 80 different answers.

Forcing them to be reproducible costs 1.6-2x speed.

µInference

It does not batch.

One request at a time, one sum order, always.

So the problem does not arise.

Reproducibility costs nothing here.

It is a result of the design.

```
for (j = 0; j < n; j++)  val += wr[j] * x[j];
```

This loop must stay exactly this shape.

Which maths operations are used

The IEEE-754 standard requires exact results for some operations, not others.

REQUIRED TO BE EXACT

`+` `−` `×` `÷` `√`

Any two correct systems must agree.

NOT REQUIRED TO BE EXACT

`exp` `log` `sin` `cos` `pow`

Results differ between systems.

So the engine only uses the exact ones. The rest were replaced:

`expf`	used by softmax and SwiGLU	replaced with a polynomial	within 1 ULP of libm
`powf cosf sinf`	used by RoPE	replaced with a lookup table	table is hashed and checked
`sqrtf`	used by RMSNorm	kept, it is exact	no change needed
`malloc, OpenMP, RNG`	not maths, but not reproducible	removed	fixed memory, one thread

How the repo is organised

One engine. Four ways to run it. The engine code is the same in all four.

mucore/ — the engine

```
mu_math.h   85   the maths
mu_core.h  133   the interface
mu_core.c  474   the transformer
```

692 lines

hosts/posix

Runs on macOS or Linux. For development.

159 lines

hosts/baremetal

Runs on ARM with no operating system.

338 lines

hosts/x86_64

Runs on Intel with no operating system.

365 lines

hosts/microkit

Runs on seL4 as one protection domain.

157 lines

The model weights are built into each image. One hash covers the code and the weights.

Result: seven systems, one hash

Hash of the raw output numbers from every step. 3,072,000 bytes.

```
9b78b92a305dc59611b5c53a04b538ae9c4ae18aea6c1fdbf6e45e9848b23bd2
```

✓	clang 21, -O2	macOS	
✓	clang 21, -O0	macOS	different optimisation level
✓	clang 22, -O2	macOS	different compiler version
✓	GCC 16, -O2	macOS	different compiler
✓	clang 18, -O2	Linux	different OS and CPU (CI)
✓	ARM, no OS	QEMU	no C library at all
✓	Intel, no OS	QEMU	different CPU maths implementation
✓	seL4 / Microkit	QEMU	the target system

The test can fail

Built the wrong way, the output changes. So the test is real.

CORRECT BUILD

```
-fno-fast-math
-ffp-contract=off
```

0 fused multiply-add

`9b78b92a…`

WRONG BUILD

```
-Ofast
-march=native
```

96 fused multiply-add

`351ebd7b…` different

Why fused multiply-add changes the answer

a*b+c as one instruction rounds once. As two instructions it rounds twice.

Comparing the text would not catch this. Comparing the numbers catches one bit.

Numbers

All measured.

692

lines of engine code

llama2.c is 973.
vLLM stack is millions.

4.13 MB

memory used

Same on all seven systems.
Fixed at build time.

161 tok/s

one CPU thread

Apple Silicon.
No BLAS, no SIMD, no threads.

1 ULP

worst maths error

Every one of the 2^32
possible inputs checked.

Compared against the original llama2.c

5 out of 5 test prompts give exactly the same text.

Replacing the maths functions did not change the output.

What is not claimed

Not tested on real hardware	Only in QEMU, an emulator.
Builds are not reproducible	The output is the same. The binary files are not identical.
Only one model tested	stories15M, 15 million parameters. Not tested at 7 billion.
No temperature sampling	Only greedy selection. Random sampling needs a fixed random seed.
No GPU support	A GPU needs signed vendor firmware and a closed compiler. Both would break the claim.
seL4 is not fully verified on ARM	Correctness and integrity are proven. Confidentiality is not. Single core only. No IOMMU.

Goals

DONE

- 692-line engine, four hosts
- Same output on seven systems
- Runs on seL4
- Proved: no memory errors (6 of 8)
- Proved: the exp error bound
- Proved: only exact maths used
- Proved: the dot product bound
- All of it checked in CI

NEXT — MONTHS

- Prove reproducibility (CompCert)
- Total error for one token
- Cover the last two functions
- Run on real hardware
- Test a larger model

LATER — RESEARCH

- Prove reproducibility as a theorem
- Prove the total error bound
- Use it to check other systems
- Move to RISC-V for stronger proofs
- Try integers instead of floats

Testing is not proving

Tests check some inputs. A proof covers all of them. Both are now in use.

STILL TESTED ONLY

`Reproducibility: 7 systems, 1 prompt.`

A good sample. Still a sample.

CompCert would make it a proof.

ALREADY PROVEN

`Memory safety. The exp bound.`

Checked by machine, all inputs.

Covers inputs never run.

One thing proofs cannot do

A proof says the program is correct for all inputs. It says nothing about one specific run.

To show what happened on a specific run you need attestation or re-running.

Proofs make the engine worth trusting. They are not a receipt.

What to prove, and what is proven

Run it with: make verify

S2	Memory limit is never exceeded	CBMC	DONE 3099 checks
S3a	Only the exact maths operations are used	LLVM IR check	DONE 5 opt levels
S4a	The exp function is within a known error	exhaustive	DONE all 2^32 inputs
S1	No memory errors, for any input	CBMC	6 of 8 functions
S3b	Output depends only on the inputs	CompCert	open
S4b	The dot product error bound	Rocq proof	DONE no axioms added
S4c	The total error for one token	combine S4a, S4b	open
S5	It computes a transformer correctly	Coq	not scheduled

10 of 10 checks pass. Four of the eight parts are done.

Plan

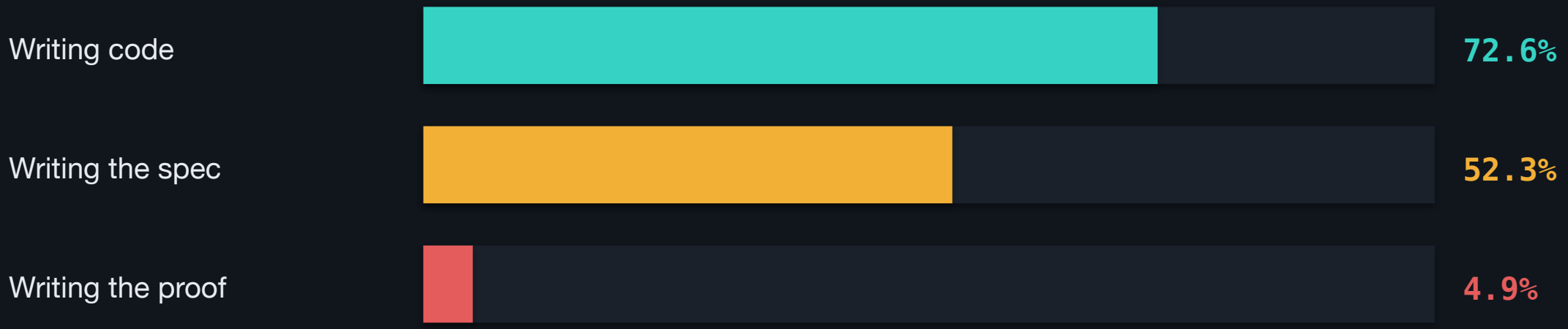
Each step is a separate piece of work with its own test in CI.

0	Run CBMC to look for crashes	done	no problems found
1	Prove no memory errors (S1, S2)	done	S2 complete, S1 6 of 8
2	Add the maths-operations check (S3a)	done	runs at 5 opt levels
3	Prove the exp error bound (S4a)	done	all 2^32 inputs, 3.4 s
4	Dot product error bound (S4b)	done	Rocq, no added axioms
5	Build with CompCert (S3b)	2–4 weeks	REPRODUCIBILITY PROVEN
6	Total error for one token (S4c)	weeks	combine S4a and S4b
7	Full correctness (S5)	research	not scheduled

Steps 0 to 4 are finished, including a hand-written Rocq proof.

What the benchmark does and does not say

VERINA measures one-shot proof writing on unseen problems. That is not this task.



Why the number is low

One attempt. Unseen problem. No compiler feedback. No choice of how to state the theorem.

S4b was written anyway

Rocq proof, no added axioms. Took about eight compile-and-fix rounds, and it caught a real error in the bound.

Next steps

1

Approve step 5: CompCert

Two to four weeks. Turns reproducibility from something tested on 7 systems into something proven for all of them. This is the publishable one.

2

Decide whether to do S5

Full correctness needs a reference transformer written in Coq. That is a large project on its own. Suggestion: leave it out for now.

3

Contact Theorem Labs

A research lab working on AI for formal verification. $6M funding. Their tools found a bug in Anthropic's sampling code that testing missed.

```
github.com/luiscosio/muInference   ·   PR #1
```